

Struct Basics

A struct (short for "structure") is a user-defined type that lets you bundle different pieces of data together into a single unit.

Think of it as creating a customized "container" or a "record" object.

If you don't have structs, managing related data becomes a nightmare. If you want to track a user's profile, you'd need separate variables for name, age, and isActive, or rely on dictionaries.

```
// The standard, messy way
name := "Alice"
age := 28
isActive := true

// Using a struct
type User struct {
    Name    string
    Age     int
    IsActive bool
}
```

Now, User acts as a single blueprint. When you create a User, you're creating one cohesive block of data.

Declaration & Zero Values

A struct is defined using the type and struct keywords, listing the field names followed by their types. Note that unlike maps, no commas separate the fields.

```
type person struct {
    name string
    age  int
}
```

If you declare a variable of this type without initializing it, Go sets every field inside the struct to its respective zero value (e.g., "" for strings, 0 for ints). var fred person // name is "", age is 0.

Initialization & Struct Literals

Go provides two styles for initializing a struct with data.

Without Field Names (Not Recommended):

You can pass a comma-separated list of values.

Drawback: You must specify a value for every field in the exact order they are declared.

With Field Names (Idiomatic):

You explicitly map values to field names.

Advantage: You can specify fields in any order, and any omitted fields simply default to their zero value. This style is much more maintainable because if you add a new field to the struct later, this code will still compile.

Once initialized, fields are read and written using standard dot notation (beth.name = "Elizabeth").

Anonymous Structs

You can declare a struct variable without first defining a named type for it.

```
pet := struct {
    name string
    kind string
}{
    name: "Fido",
    kind: "dog",
}
```

When to use them: Anonymous structs are highly idiomatic in two scenarios:

- 1. **Data Serialization:** When unmarshaling external data (like a JSON payload) into a one-off structure.
- 2. **Testing:** When writing "table-driven tests," an anonymous struct is typically used to hold the slice of test cases (inputs and expected outputs).

Comparing and Converting Structs

Comparability: Structs are comparable using == and != if and only if all of their underlying fields are comparable. If a struct contains a slice, map, channel, or function, it is not comparable.

Conversion: You cannot use == between two structs of different types. However, you can explicitly cast one struct to another (e.g., secondPerson(f)) if both structs have the exact same field names, types, and order.

Anonymous Exception: If you compare or assign a named struct to an anonymous struct, Go allows it directly without requiring an explicit type cast, provided the fields match perfectly.

Pitfalls & Fixes

1: Comparing Structs Incorrectly

The Mistake: Developers try to use the == operator on structs that contain uncomparable fields, such as slices.

```
type customer struct {
    id          string
    operations []float64 // Slice makes the struct uncomparable
}

cust1 := customer{id: "x", operations: []float64{1.}}
cust2 := customer{id: "x", operations: []float64{1.}}
fmt.Println(cust1 == cust2) // COMPILER ERROR
```

Attempting to compare cust1 == cust2 will result in a compile-time error. Furthermore, if you accidentally assign these structs to any interface variables and compare them, the code will compile but cause a runtime panic.

The Fix:

- 1. For runtime checks: Implement a custom equal() method on the struct that manually compares the slices via a loop or use reflect.DeepEqual() (though reflection is much slower).
- 2. For testing: Use reflect.DeepEqual() or standard testing utilities like go-cmp to compare the structs safely.

2: Memory Bloat via Data Alignment

The Concept:

CPUs read memory in blocks (words), which are typically 8 bytes on a 64-bit architecture. To ensure fast memory access, the Go compiler relies on "data alignment" by padding structs.

The Mistake:

Declaring struct fields without considering their sizes can bloat your memory usage.

```
type Foo struct {
    b1 byte    // 1 byte
    i   int64   // 8 bytes
    b2 byte    // 1 byte
}
```

Because the int64 cannot be split across word boundaries, the compiler injects 7 bytes of invisible "padding" after b1 and another 7 bytes after b2. As a result, this struct consumes 24 bytes of memory even though the actual data is only 10 bytes.

If you allocate a slice of one million of these structs, you are wasting megabytes of memory and increasing the garbage collector's workload.

The Fix:

Always sort your struct fields by type size in descending order.

```
type Foo struct {
    i   int64   // 8 bytes
    b1 byte    // 1 byte
    b2 byte    // 1 byte
}
```

By putting the 8-byte field first, b1 and b2 can sit side-by-side in the next memory block. The compiler only needs to pad the end. This simple reordering shrinks the struct from 24 bytes to 16 bytes.